SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA0301
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO4: *Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. (BL3, BL4, BL5)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Technical Presentation **DURATION: 2HRS**

1**. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning**

- **Bellman Principle of Optimality**
- **Policy Evaluation and Policy Improvement**
- **Value Iteration vs. Policy Iteration**
- **Industrial applications**

(25 Marks)

1. Bellman Principle of Optimality

Dynamic Programming (DP) is a powerful approach used in Reinforcement Learning (RL) to solve sequential decision-making problems. It divides a complex problem into smaller subproblems and solves them systematically. The main idea behind DP in RL is based on the Bellman Principle of Optimality.

The Bellman Principle states that:*An optimal policy has the property that, whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy.*

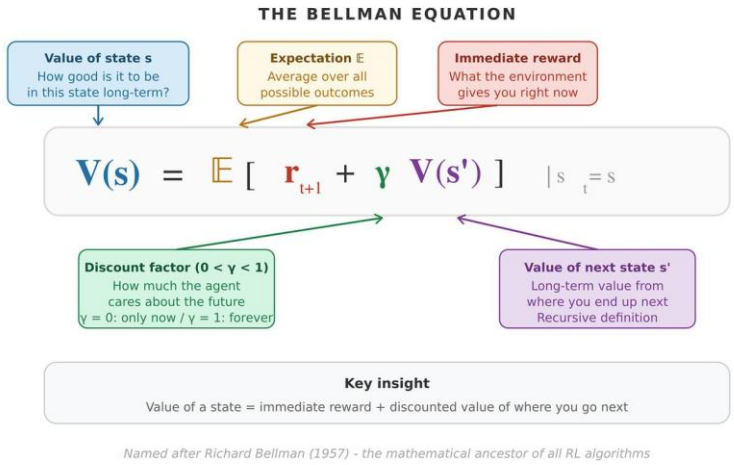
In RL, an agent makes a sequence of decisions. For example, a robot may need to decide whether to move forward, turn left, or turn right to reach a target. Instead of calculating the complete sequence at once, Dynamic Programming calculates the value of each state and uses these values to select the best action.

The Bellman optimality equation is:

$$V^*(s) = \max_a [\, R(s,a) + \gamma \Sigma P(s'|s,a)V^*(s') \,]$$

Where:

- **$V^*(s)$** = optimal value of state *s*
- **$R(s,a)$** = reward received after taking action *a*
- **γ** = discount factor
- **$P(s'|s,a)$** = probability of reaching next state
- **$V^*(s')$** = value of the next state



This principle allows the agent to determine the best decision by considering both the immediate reward and the future rewards.

2. Policy Evaluation and Policy Improvement

A **policy** defines the strategy followed by an RL agent. It specifies which action should be selected in each state.

Policy Evaluation calculates how good a particular policy is. It estimates the expected total future reward when the agent follows that policy.

For example, suppose a robot follows a policy that says:

State → Action → Next State → Reward

The value of every state is repeatedly updated until the values become stable.

After evaluation, the next step is **Policy Improvement**. The agent checks whether choosing a different action could provide a better expected reward. If a better action is found, the policy is updated.

The process can be represented as:

Initial Policy → Policy Evaluation → Policy Improvement → New Policy → Repeat

The process continues until no action can improve the policy. At that point, the agent has reached an **optimal policy**.

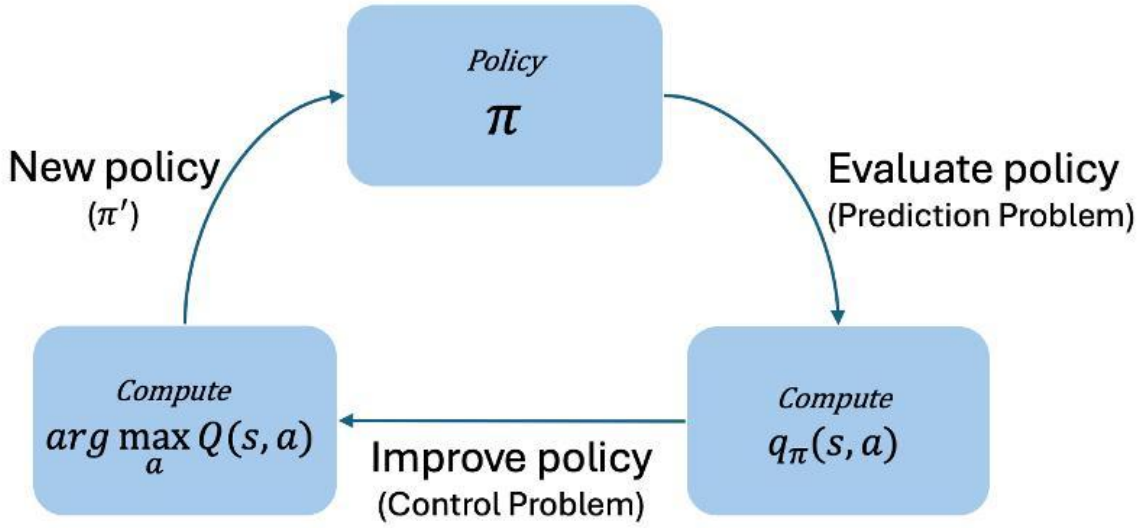
3. Value Iteration vs. Policy Iteration

Two major Dynamic Programming methods used in RL are **Value Iteration** and **Policy Iteration**.

Feature	Value Iteration	Policy Iteration
Main idea	Directly finds optimal state values	Alternates between evaluation and improvement
Policy	Extracted after value convergence	Improved during each iteration
Speed	Often faster per iteration	May require more computation per iteration
Process	Value update → Optimal policy	Policy evaluation → Policy improvement
Suitable for	Problems where direct value optimization is useful	Problems with manageable state/action spaces

Value Iteration repeatedly applies the Bellman optimality equation until the value function converges. The optimal action is then selected from each state.

Policy Iteration starts with an initial policy, evaluates it, improves it, and repeats these two steps until the policy becomes stable.



4. Industrial Applications

Dynamic Programming and RL are useful in many real-world industrial applications because they help systems make decisions while considering **long-term consequences**.

1. Robotics:
Robots use sequential decision-making to determine efficient paths, avoid obstacles, and perform tasks automatically.

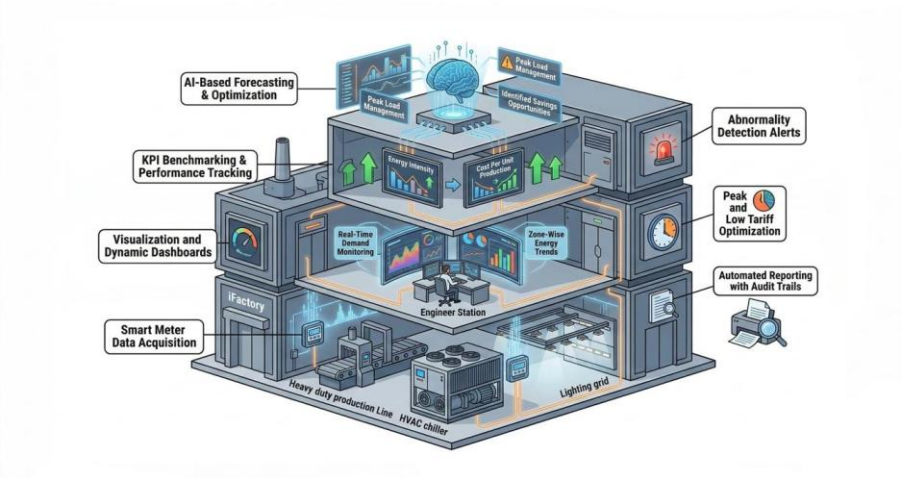
2. Manufacturing:
RL can optimize production schedules, machine usage, and resource allocation to reduce production time and cost.

3. Autonomous Vehicles:
Vehicles can use decision-making techniques to select actions such as changing lanes, accelerating, braking, and avoiding obstacles.

4. Supply Chain Management:
Companies can optimize inventory levels, transportation routes, and warehouse operations by considering future demand and costs.

5. Energy Management:
Power systems can determine when to store, distribute, or consume energy to reduce wastage and operational costs.

6. Healthcare:
Sequential decision-making can assist in treatment planning by selecting actions based on the patient's changing condition.



Conclusion

Dynamic Programming provides the mathematical foundation for solving many sequential decision-making problems in Reinforcement Learning. Bellman's Principle of Optimality breaks a complex decision problem into smaller subproblems. Policy Evaluation and Policy Improvement help an agent gradually discover better strategies, while Value Iteration and Policy Iteration provide systematic methods for finding optimal policies. These techniques have applications in robotics, manufacturing, autonomous vehicles, supply chains, healthcare, and energy management.

2. Monte Carlo Methods and Temporal-Difference Learning: A Comparative Study

- **Monte Carlo Prediction and Control**
- **TD(0), SARSA, and Q-Learning**
- **Sample efficiency**
- **Performance comparison through experiments**

(25 Marks)

1. Monte Carlo Prediction and Control

Monte Carlo (MC) methods are reinforcement learning techniques that learn from the actual experience of an agent. Unlike Dynamic Programming, Monte Carlo methods do not require a complete model of the environment. They learn directly from episodes of interaction.

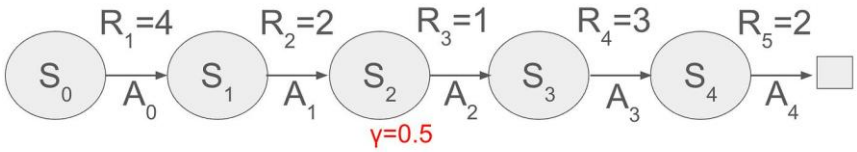
In Monte Carlo Prediction, the value of a state is estimated by observing the total reward obtained after visiting that state until the episode ends.

For example, if an agent plays a game and receives rewards of +10, +5, and +20, the return from an earlier state can be calculated using these rewards. By averaging returns from many episodes, the agent estimates the value of that state.

The basic idea is:

State → Actions → Rewards → Episode Ends → Calculate Return → Update Value

Monte Carlo Control extends this idea to find an optimal policy. The agent explores different actions, observes the final rewards, and gradually learns which actions provide better long-term returns.



$G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \gamma^4 R_5 = R_1 + \gamma(G_1) = 4 + 0.5*3.5 = 5.75$

$G_1 = R_2 + \gamma R_3 + \gamma^2 R_4 + \gamma^3 R_5 = R_2 + \gamma(G_2) = 2 + 0.5*3 = 3.5$

$G_2 = R_3 + \gamma^2 R_4 + \gamma^3 R_5 = R_3 + \gamma(G_3) = 1 + 0.5*4 = 3$

$G_3 = R_4 + \gamma R_5 = R_4 + \gamma(G_4) = 3 + 0.5*2 = 4$

$G_4 = R_5 = 2$

2. Temporal-Difference Learning: TD(0), SARSA and Q-Learning

Temporal-Difference (TD) learning combines ideas from Monte Carlo methods and Dynamic Programming. The major advantage is that TD methods can **learn before an episode is completed**.

TD(0)

TD(0) updates the value of a state using the reward obtained immediately and the estimated value of the next state.

$$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$$

Here, α is the learning rate and γ is the discount factor.

This makes TD(0) useful when environments have long or continuous episodes.

SARSA

SARSA is an **on-policy** TD control algorithm. Its name comes from the sequence:

State → Action → Reward → State → Action

The algorithm updates the action-value function based on the action that the current policy actually selects.

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S',A') - Q(S,A)]$$

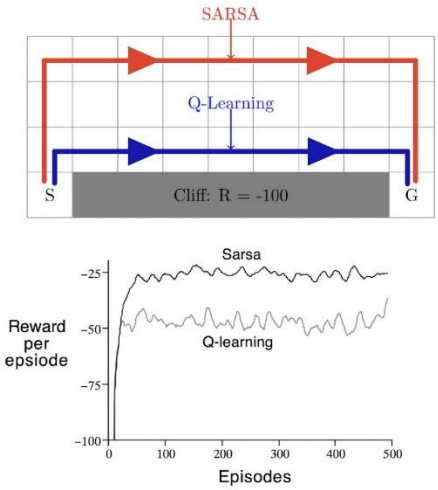
Q-Learning

Q-Learning is an **off-policy** TD algorithm. It learns the value of the best possible action even if the agent follows an exploratory policy.

Its update rule is:

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max Q(S',a) - Q(S,A)]$$

Q-Learning is widely used because it can learn an optimal policy through trial and error.



3. Sample Efficiency

Sample efficiency refers to how effectively an algorithm learns from a limited number of experiences.

Monte Carlo methods generally require a **complete episode** before updating values. Therefore, they may need more samples before learning becomes effective.

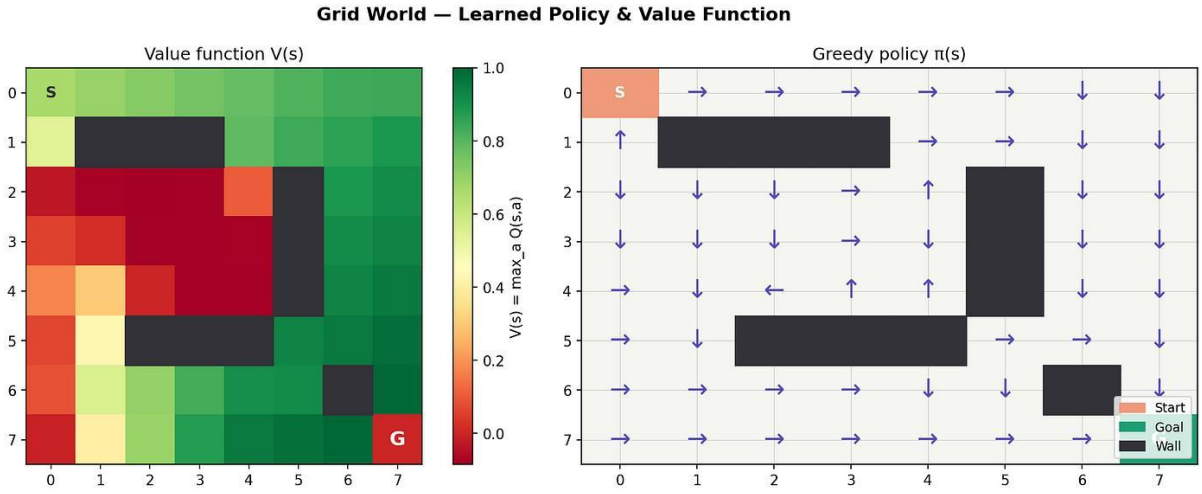
TD methods can update their estimates **after every step**, making them more sample-efficient in many environments.

Method	Learning Update	Sample Efficiency
Monte Carlo	After episode ends	Moderate
TD(0)	After each step	High
SARSA	After each step	High
Q-Learning	After each step	High

For example, imagine a robot learning to reach a destination. A Monte Carlo method may wait until the robot reaches the destination before evaluating the complete experience. A TD algorithm can learn after every movement, allowing it to improve its behavior much faster.

4. Performance Comparison Through Experiments

A simple experiment can be conducted using a **Grid World environment**, where an agent must move from a starting position to a goal while avoiding obstacles.



Suppose the algorithms are trained for **500 episodes**. Their performance can be compared using **average reward, number of steps required to reach the goal, and convergence speed**.

Method	**Learning Speed**	**Average Reward**	**Convergence**
Monte Carlo	Moderate	Good	Slower
TD(0)	Fast	Good	Fast
SARSA	Fast	Very Good	Stable
Q-Learning	Very Fast	Excellent	Fast

In many simple environments, **Q-Learning** can achieve high rewards because it directly learns the best action for each state. **SARSA** can provide safer behavior because it learns according to the actions actually taken by the current policy. Monte Carlo methods can perform well but may require more complete episodes to obtain useful estimates.

Conclusion

Monte Carlo and Temporal-Difference methods are important approaches for learning without explicitly knowing the environment model. **Monte Carlo methods** learn from complete episodes, while **TD methods learn continuously from step-by-step experience**. TD(0), SARSA, and Q-Learning are generally more sample-efficient because they update values before an episode ends. In experimental environments such as Grid World, Q-Learning often provides strong performance, while SARSA offers stable learning and Monte Carlo provides a simple episode-based learning approach.

3. Deep Q-Networks for Intelligent Decision-Making

- **DQN Architecture**
- **Experience Replay**
- **Target Networks**
- **Industrial applications in robotics and autonomous systems**

(25 Marks)

1. DQN Architecture

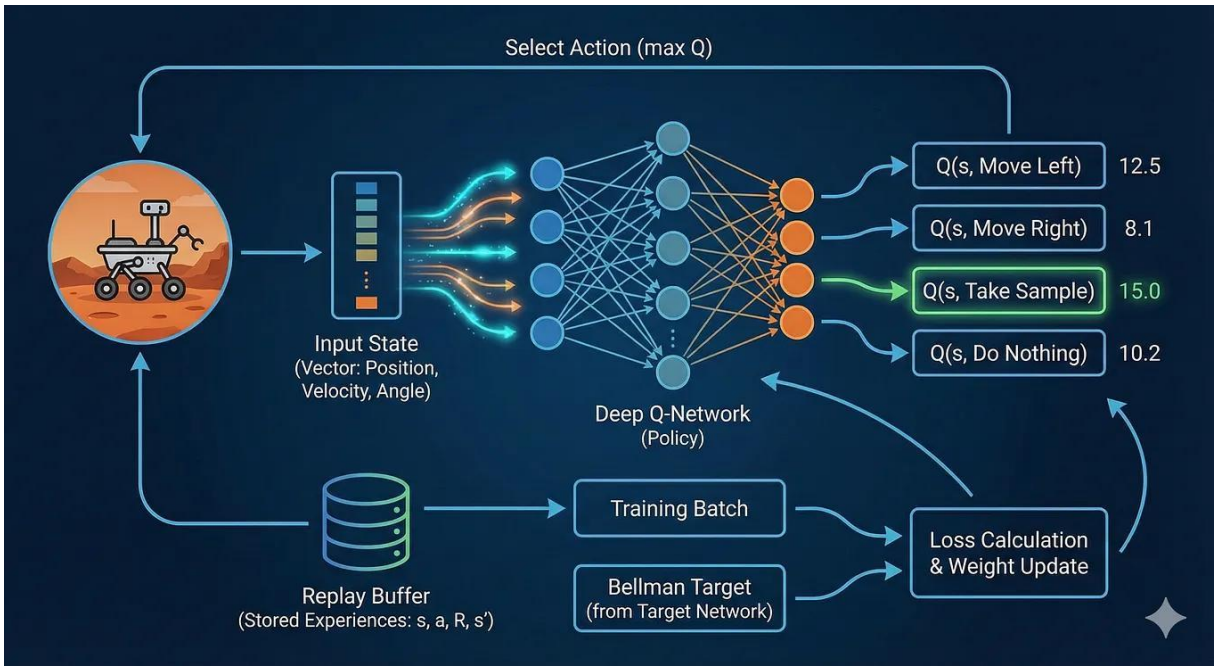
A Deep Q-Network (DQN) is a reinforcement learning algorithm that combines Q-Learning with a Deep Neural Network. It is designed to handle environments with a large number of states, where maintaining a traditional Q-table becomes impractical.

In DQN, the neural network receives the current state as input and produces a Q-value for each possible action. The agent then selects the action with the highest Q-value, while sometimes choosing random actions for exploration.

The basic structure is:

Environment → State → Deep Neural Network → Q-values → Best Action → Environment

For example, in a robotic system, the state may contain information from cameras and sensors. The DQN processes this information and decides whether the robot should move forward, turn, stop, or change direction.



6

A typical DQN consists of:

- **Input Layer:** Receives the state of the environment.
- **Hidden Layers:** Extract important features using neural network operations.
- **Output Layer:** Produces Q-values corresponding to possible actions.
- **Action Selection:** The action with the highest Q-value is selected.

2. Experience Replay

One of the important features of DQN is **Experience Replay**. During interaction with the environment, the agent stores its experiences in a memory buffer.

Each experience is represented as:

(State, Action, Reward, Next State, Done)

Instead of learning only from the most recent experience, DQN randomly selects a **mini-batch of past experiences** from the replay memory and uses them for training.

Experience Replay provides two major benefits:

1. **Breaks correlation between consecutive experiences:** Consecutive observations are often similar. Random sampling makes training data more independent.
2. **Reuses previous experiences:** The same experience can be used multiple times, improving sample efficiency.

For example, if a robot previously learned that an obstacle should be avoided, that experience can be stored and reused during future training.

3. Target Networks

Another important component of DQN is the **Target Network**. Using the same network to calculate both the current Q-value and the target Q-value can make training unstable.

Therefore, DQN uses two networks:

1. Online Network: Continuously updated during training.

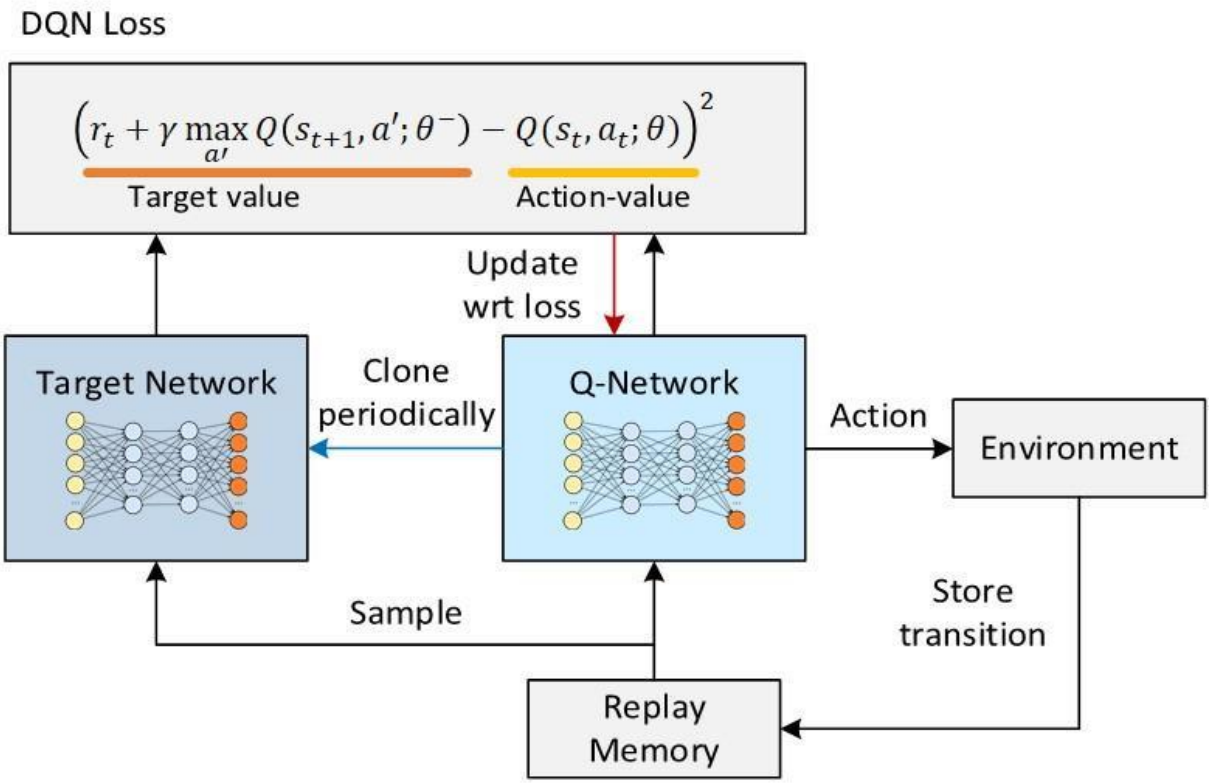
2. Target Network: Used to calculate stable target Q-values.

The target network is periodically updated using the weights of the online network rather than changing after every training step.

The target value is commonly calculated as:

Target = R + γ max Q_target(S', a')

The difference between the predicted Q-value and the target value is used to calculate the **loss**, which is then used to update the online network.



This separation makes the learning process more stable and reduces unwanted fluctuations during training.

4. Industrial Applications in Robotics and Autonomous Systems

DQN has applications in systems that require **intelligent decision-making in complex environments**.

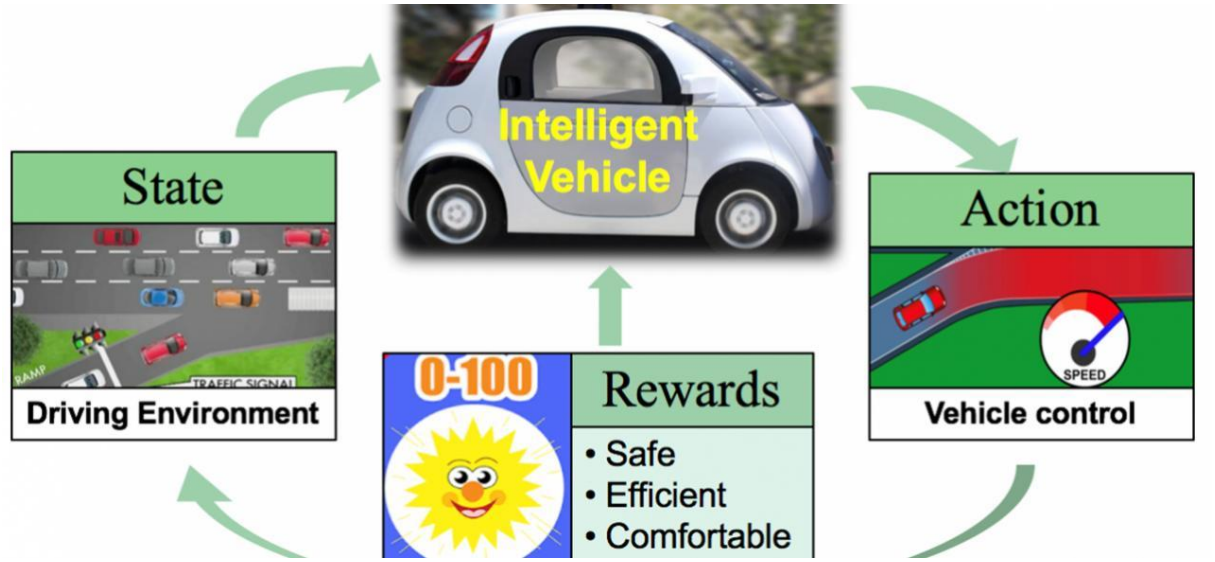
Robotics:
DQN can help robots learn navigation, obstacle avoidance, path planning, and task execution. A robot can learn which action to take based on camera and sensor information.

Autonomous Vehicles:
DQN can support decisions such as lane selection, acceleration, braking, and obstacle avoidance. The system can learn suitable actions by interacting with simulated or real environments.

Industrial Robots:
Robotic arms can use reinforcement learning to learn efficient movement sequences for tasks such as object picking, assembly, and sorting.

Drones:
DQN can help drones select navigation actions, avoid obstacles, and reach specified destinations.

Warehouse Automation:
Autonomous mobile robots can use intelligent decision-making to select routes for transporting goods while avoiding other robots and obstacles.



Conclusion

Deep Q-Networks combine **deep learning and reinforcement learning** to solve complex decision-making problems. The **DQN architecture** estimates Q-values using a neural network, while **Experience Replay** improves learning by reusing past experiences. **Target Networks** provide stability during training. These techniques make DQN particularly useful for robotics, autonomous vehicles, drones, and industrial automation, where intelligent decisions must be made continuously in dynamic environments.

4. Comparative Analysis of Advanced Deep Q-Learning Algorithms

- **Double DQN (DDQN)**
- **Dueling DQN**
- **Prioritized Experience Replay (PER)**
- **Performance comparison using benchmark environments**

(25 Marks)

1. Double DQN (DDQN)

Double Deep Q-Network (DDQN) is an improved version of the standard DQN algorithm. One major problem in DQN is overestimation of Q-values. DQN uses the same network to select the best action and estimate its value, which can sometimes produce unrealistically high Q-values.

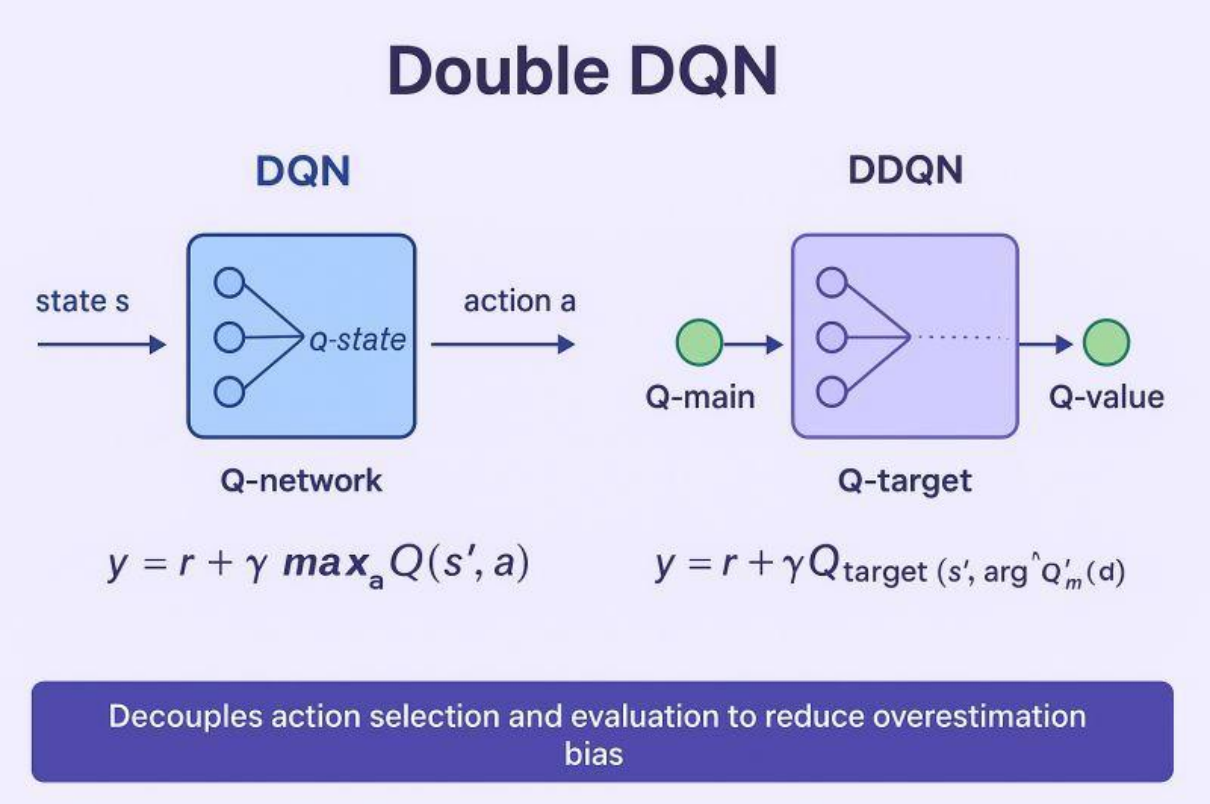
DDQN solves this problem by separating these two tasks:

- **Online network:** Selects the best action.
- **Target network:** Estimates the value of the selected action.

The basic idea is:

Select action using Online Network → Evaluate action using Target Network → Update Q-value

This reduces overestimation and generally provides **more stable learning** than standard DQN.



2. Dueling DQN

Dueling DQN modifies the architecture of DQN by separating the estimation of **state value** and **action advantage**.

Instead of directly producing Q-values, the network has two streams:

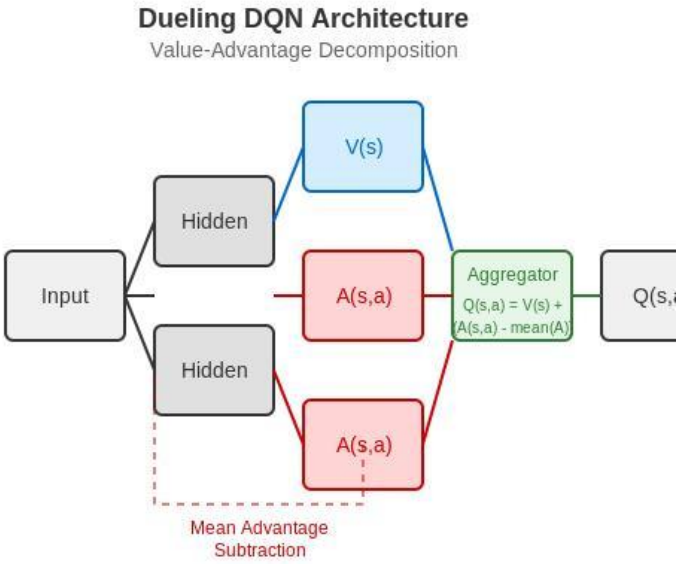
1. **Value Stream:** Estimates how good the current state is.
2. **Advantage Stream:** Estimates how much better one action is compared with other actions.

These streams are combined to calculate the final Q-value.

Q(s,a) = V(s) + A(s,a)

Dueling DQN is particularly useful when many actions have similar outcomes. For example, in a navigation problem, the agent may be in a state where several actions are equally safe. The

network can recognize that the **state itself is valuable**, even when the difference between actions is small.



3. Prioritized Experience Replay (PER)

Standard DQN uses random Experience Replay, where experiences are selected randomly from the replay memory. However, not all experiences are equally useful.

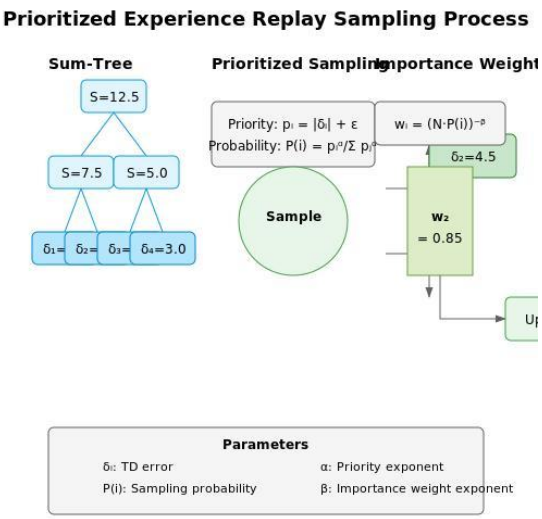
Prioritized Experience Replay (PER) gives higher priority to experiences from which the agent can learn more.

Experiences with a larger TD error generally receive higher priority because they indicate that the current prediction differs significantly from the expected value.

The process is:

Store Experiences → Calculate TD Error → Assign Priority → Select Important Experiences → Train Network

For example, if an autonomous robot unexpectedly receives a large negative reward after hitting an obstacle, that experience is highly informative. PER gives it a higher probability of being selected during training.



4. Performance Comparison Using Benchmark Environments

Advanced DQN algorithms can be compared using benchmark environments such as CartPole, LunarLander, and Atari games. Important performance measures include:

- Average reward
- Learning speed
- Training stability
- Sample efficiency
- Convergence performance

A general qualitative comparison is:

Algorithm	**Main Improvement**	**Learning Stability**	**Sample Efficiency**
DQN	Basic deep Q-learning	Moderate	Moderate
DDQN	Reduces Q-value overestimation	High	Good
Dueling DQN	Separates value and advantage	High	Good
PER	Learns more from important experiences	High	Very Good

Qualitative comparison of advanced DQN methods

Illustrative scores showing the typical relative strengths of the methods; these are not measured benchmark results.

0255075100DQNDDQNDueling DQNPER

These values are illustrative rather than experimental measurements. In an actual experiment, the algorithms should be trained under identical conditions and compared using the same environment, number of episodes, random seeds, and evaluation metrics.

Conclusion

DDQN, Dueling DQN, and PER address different limitations of standard DQN. DDQN reduces overestimation of Q-values, Dueling DQN improves the representation of state values and action advantages, and PER focuses learning on more informative experiences. These techniques can also be combined—for example, a system can use Dueling DDQN with Prioritized Experience Replay—to obtain a more efficient and stable deep reinforcement learning agent.

5. Policy-Based Reinforcement Learning for Continuous Control Problems

- **Vanilla Policy Gradient**
- **REINFORCE Algorithm**
- **Stochastic Policy Search**
- **Applications in robotics and autonomous driving**

(25 Marks)

1. Vanilla Policy Gradient

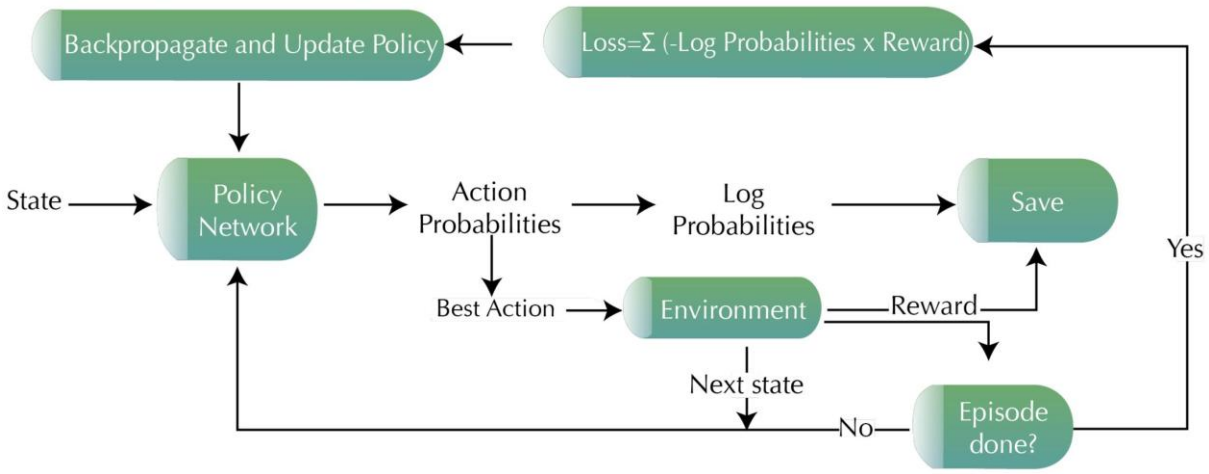
Policy-Based Reinforcement Learning directly learns a policy instead of first learning a value function or Q-table. A policy determines the probability of selecting an action in a particular state.

This approach is especially useful for continuous control problems, where actions can have continuous values. For example, a robotic arm may need to decide the exact angle of a joint rather than simply choosing between "left" and "right."

In Vanilla Policy Gradient, a neural network represents the policy. The network receives the current state and produces a probability distribution over possible actions. The policy parameters are then adjusted so that actions producing higher rewards become more likely.

The basic learning process is:

State → Policy Network → Action → Environment → Reward → Update Policy



The main advantage of policy-gradient methods is that they can naturally handle **continuous action spaces** and stochastic policies.

2. REINFORCE Algorithm

REINFORCE is one of the simplest and most widely known policy-gradient algorithms. It learns directly from complete episodes of experience.

The algorithm follows these steps:

1. Initialize the policy parameters.
2. Start an episode from an initial state.
3. Select actions according to the current policy.
4. Continue until the episode ends.
5. Calculate the total return obtained from the episode.
6. Increase the probability of actions that produced high rewards.
7. Repeat the process for many episodes.

The policy is updated according to the relationship between the **return received** and the **probability of selecting an action**.

For example, if a robot successfully reaches a target after taking a particular sequence of movements, REINFORCE increases the probability of making similar decisions in future episodes.

Advantages:

- Simple concept and implementation.
- Works with continuous action spaces.
- Directly optimizes the policy.

Limitations:

- Can have high variance in learning.
- Usually requires many episodes.
- Learning can be slower than more advanced actor-critic methods.

3. Stochastic Policy Search

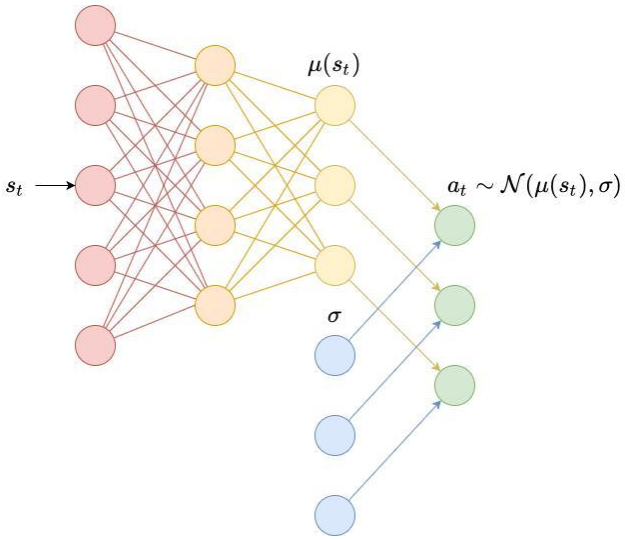
A **stochastic policy** does not always select the same action for a given state. Instead, it generates a probability distribution over possible actions.

For example:

State → Policy → 70% Action A, 20% Action B, 10% Action C

This randomness is useful because it encourages **exploration**. The agent can try different actions and discover better strategies instead of always following the same behavior.

In continuous control, the policy can generate parameters of a probability distribution, such as the **mean and standard deviation of a Gaussian distribution**. An action can then be sampled from this distribution.



Stochastic policy search is useful when the environment is uncertain or when multiple possible actions can lead to successful outcomes.

4. Applications in Robotics and Autonomous Driving

Policy-based RL is highly suitable for real-world systems where actions are continuous.

Robotics:
A robotic arm may need to control the exact movement of its joints, while mobile robots may continuously control their speed and direction. Policy-gradient methods can learn these movements through interaction with the environment.

Robot Navigation:
A robot can learn how to adjust its velocity and steering angle to reach a destination while avoiding obstacles.

Autonomous Driving:
An autonomous vehicle needs continuous control over **steering, acceleration, and braking**. A policy network can learn appropriate control actions based on information from cameras, sensors, and the vehicle's current state.

Drone Control:
Policy-based methods can help drones learn continuous changes in thrust, direction, and movement for stable flight and navigation.

Conclusion

Policy-Based Reinforcement Learning directly learns the best strategy for selecting actions. **Vanilla Policy Gradient** provides a basic approach for optimizing policy parameters, while **REINFORCE** learns by using complete episodes and their returns. **Stochastic Policy Search**

introduces controlled randomness, helping the agent explore different actions. These methods are particularly useful for continuous-control applications such as **robotic arms, autonomous robots, drones, and autonomous vehicles**, where actions are not limited to a small set of discrete choices.

Code of Conduct:

I Malli Chandana(192472250)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student : Malli Chandana

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness